AI Compiler Performance Engineering

Auto-Loops Books — ch16

Contents

1 TVM and Apache TVM Stack **1**

1.1 TVM HW matrix 1

1.2 TVM arch passes 4

1.3 TVM codegen 6

1.4 TVM tuning 9

1.5 TVM case study 12

1.6 Key Takeaways 16

1.7 Conclusion 17

Bibliography **18**

Index **20**

Chapter 1

TVM and Apache TVM Stack

Apache TVM made one bet that still defines the stack: separate *what* a tensor operator computes from *how* it is scheduled onto silicon, then let a learning-based cost model search the schedule space instead of a vendor library author (Chen *et al.*, 2018b). That decoupling is why a single operator definition can lower to x86, ARM, CUDA, and FPGA back-ends without rewriting the math (Chen *et al.*, 2018b; Chen, 2020). It is also why teams misread TVM in production: they tune a model until prefill GEMMs look excellent, ship it, and then watch interactive batch-1 decode miss p99 anyway (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023). The reason is the same one Chapter 1 established. Dense Llama-3.2-1B averages 847.5 GPU kernel launches per output token on H100 (BS=4, SL=2048); OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). A correct, well-tuned TVM operator schedule does nothing about that launch storm unless the compiler is also asked to fuse and capture the decode loop (Chen, 2026; NVIDIA, 2024). This chapter treats TVM as a production residency contract under decode metrics—bytes/token and launches/token—not as a tutorial on schedule primitives (Davies *et al.*, 2025).

1.1 TVM HW matrix

TVM's selling point is a hardware/target support **matrix**: write the compute once, lower it to many back-ends (Chen *et al.*, 2018b). In practice that matrix is necessary but not sufficient for decode. A target that *builds* is not a target that is decode-legal, and most "mysterious" regressions come from treating the two as the same thing (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

The hardware truth underneath the matrix is the usual split. GPUs bound a

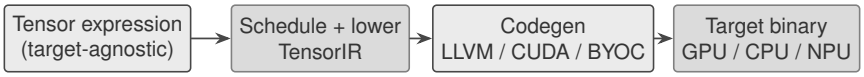


Figure 1.1: One tensor-expression definition fans out across the TVM hardware/target matrix: schedule and lower to TensorIR, then emit per-backend code.

schedule by shared-memory capacity and tensor-core feeding; CPUs by cache blocking and NUMA locality; NPUs and XDNA/AIE fabrics by static tile SRAM and DMA chains that must be assigned at compile time (NVIDIA, 2022; AMD, 2024b; Goodfellow *et al.*, 2016). At batch-1, decode stays bandwidth- and orchestration-limited on all three—the per-token matmul is too thin to amortize either HBM traffic or launch overhead (Chen, 2026; Vellaisamy *et al.*, 2026).

TVM earns the matrix because the same tensor expression carries no target assumptions: the compute is declared once, and a separate schedule chooses tiling, vectorization, thread binding, and memory scope per target (Chen *et al.*, 2018b). That is genuinely more than vendor libraries deliver, where each new SKU or accelerator demands a hand-written kernel (Chen, 2020). The catch for serving is that the matrix is graded at *build* time—does the target lower and run—while decode goodput is a *runtime* property of bytes moved and kernels launched. A green build matrix cell tells you the operator exists on that target; it tells you nothing about whether the lowered schedule keeps the decoder layer on-chip.

Multi-hardware delta. The same TVM operator definition takes a different legal schedule per class: a Hopper GPU wants TMA-fed double buffering and a captured launch graph; an x86 CPU wants cache-blocked tiles and SIMD; an XDNA NPU wants a static tile-to-DMA assignment with no dynamic dispatch at all (SemiAnalysis, 2024; AMD, 2024b,a). Identical math, three different bytes/token outcomes. This is exactly the residency reasoning YiRage encodes as checkable IR metadata so the matrix entry is rejected before it ships, not after p99 slips (Chen and YiRage Contributors, 2026).

1.1.1 Entering TVM from serving frameworks

The matrix is only reachable through an import path, and the import path decides how much of a decode loop TVM sees (Chen *et al.*, 2018b; Lai *et al.*, 2023). Relay and Relax both accept models from PyTorch, ONNX, TensorFlow, and native front ends, but the granularity of the exported graph determines what can later be fused and scheduled (Chen *et al.*, 2018b; Lai *et al.*, 2023). A PyTorch export that keeps layer-norm, linear, attention, and MLP as separate framework ops gives

the graph passes the seams they need; an export that already bakes everything into opaque custom calls hides those seams forever (Davies *et al.*, 2025; Chen, 2020). Teams should therefore verify the imported graph before tuning: count the top-level operators per decoder layer, confirm that dynamic shapes are represented rather than padded away silently, and confirm that stateful sampling kernels stayed outside the module (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Import quality is a compile-time property with runtime consequences.

1.1.2 Graph granularity and decoder layers

The unit TVM compiles is the whole imported module, but the unit that matters for decode is one decoder layer repeated many times (Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). A well-structured import lets the graph passes identify the repeating layer pattern and compile it once as a fused call; a flat import treats every layer as unique and multiplies compile and tuning cost without adding any new information (Chen *et al.*, 2018b; Davies *et al.*, 2025). Relax improves on Relay here by keeping dynamic shapes and memory planning at graph level, so a decoder loop with varying KV length can be represented as one dynamic function rather than dozens of recompiled instances (Lai *et al.*, 2023; Kwon *et al.*, 2023). For production, the recommended practice is to export one decoder layer as a named function, tune and fuse it once, and let the serving runtime invoke that compiled function per token (Davies *et al.*, 2025; Chen, 2026). This keeps the compiler's work aligned with the unit of work the serving loop actually repeats.

1.1.3 Matrix versioning per SKU

The build matrix has a versioning problem that decode teams discover late: TVM's legal schedules change with the TVM release, the target description, and the host toolchain, so a matrix cell that was green six months ago says little about today (Chen *et al.*, 2018b; Chen, 2020). Each cell should carry the compiler version, target string, runtime executor, and the measured decode counters from the last validation run (Chen, 2026; Vellaisamy *et al.*, 2026). A build matrix with dates and artifact hashes turns "TVM supports XDNA" into a reproducible claim instead of a documentation promise (AMD, 2024b; Chen and YiRage Contributors, 2026). CI should regenerate the matrix on compiler upgrades and mark any cell whose decode bytes/token regressed before the release is promoted (Davies *et al.*, 2025; Chen, 2026).

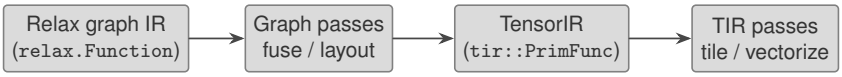


Figure 1.2: TVM's two-level pass pipeline: graph-level Relax transforms feed loop-level TensorIR transforms inside one `IRModule`.

1.1.4 Target qualification cells

Because a matrix cell that builds is not a cell that serves, each supported target needs a qualification record with three parts: the build test (does the target lower and run the operator), the legality test (do the emitted schedules respect on-chip budgets and static constraints), and the decode-cell benchmark (what bytes/token and launches or DMA edges the artifact produces at served context buckets) (Chen *et al.*, 2018b; AMD, 2024b; Chen, 2026). The legality test is where TVM differs most from vendor libraries: TVM's schedule primitives are powerful enough to emit an illegal-for-decode kernel that still passes unit tests, so the qualification record must review the schedule, not only the result (AMD, 2024a; Chen and YiRage Contributors, 2026; Chen, 2020). CPU and GPU cells can be qualified with the same benchmark script; an NPU cell needs the script extended with static-allocation and DMA-edge checks (AMD, 2024b; Davies *et al.*, 2025). The matrix becomes trustworthy when every green cell carries the three artifacts and a date (Vellaisamy *et al.*, 2026; Chen, 2026).

1.2 TVM arch passes

TVM runs optimization at two IR levels, and the production surprise is almost never a missing peak-FLOPs pass—it is an **optim**ization that is legal in isolation but spills a fused producer to HBM between levels (Chen *et al.*, 2018b; Chen and YiRage Contributors, 2026). The modern stack makes the two levels explicit. A graph-level `relax.Function` carries the dataflow; a `tir::PrimFunc` carries the loop nest; both live in one `IRModule`, and `call_tir` lowers a graph node into a loop-level kernel in destination-passing style so intermediate buffers can be planned globally (Lai *et al.*, 2023). Relax replaced the older Relay model precisely because Relay treated tensor programs as opaque foreign functions and could not track dynamic shapes—or residency—across the boundary (Lai *et al.*, 2023).

This is a deliberately different choice from an open dialect tower. Where MLIR exposes many progressively lowered *dialect*s, TVM commits to a fixed graph→TensorIR pair and pushes its leverage into schedule search rather than

dialect proliferation (Chen *et al.*, 2018b; Lai *et al.*, 2023). The engineering consequence: graph passes like operator fusion must be co-designed with the TensorIR **lower**ing that follows, or fusion "succeeds" on paper while the lowered kernel still round-trips activations through DRAM.

Concretely, the graph-level passes that decide decode residency are a short list, and each has a loop-level consequence. `FuseOps` merges a chain of element-wise and reduction operators into one `call_tir` target so the lowered kernel keeps intermediates in registers or shared memory instead of HBM (Chen *et al.*, 2018b). `ToMixedPrecision` and layout-rewrite passes change how many bytes each activation costs and whether the tensor-core or vector path is even reachable on the target (Lai *et al.*, 2023). `StaticPlanBlockMemory` and the destination-passing-style contract behind `call_tir` let the compiler allocate one intermediate buffer and reuse it across the fused region, which is the difference between a decoder layer that streams its working set once and one that re-reads it per operator (Lai *et al.*, 2023; Chen, 2026). Run these in the wrong order—layout after fusion, say—and the fused kernel is forced back through a transpose that re-materializes the very activations fusion was meant to keep on-chip.

What to diff before merge. Lower the same module twice and diff the generated TIR (or CUDA) for new buffer allocations; count launches/token on the GPU path and DMA edges on the NPU path (Vellaisamy *et al.*, 2026; AMD, 2024a). A silent spill introduced by a reordered pass reads downstream as an unexplained p99 regression, and graph-capture legality must hold on the GPU path while static-allocation legality must hold on XDNA (NVIDIA, 2024; AMD, 2024b).

1.2.1 Fusion boundaries at the graph level

Graph-level fusion in TVM must answer the same legality question as the theory chapter: which intermediates can stay on-chip between producer and consumer (Chen, 2020, 2026). `FuseOps` works on chains where layout and data type match, and it stops at reduction boundaries, custom calls, and dynamic-shape operations that the scheduler cannot reason about (Chen *et al.*, 2018b; Lai *et al.*, 2023). Decode graphs are full of such stops: online softmax state crosses a reduction boundary inside attention, and attention output feeds a layout-sensitive MLP (Dao *et al.*, 2022; Davies *et al.*, 2025). The practical discipline is to enumerate those stops in the imported graph and decide which are intentional (custom kernels, routing) and which are import artifacts that should be fixed before compilation (Chen *et al.*, 2018b; Chen and YiRage Contributors, 2026). Every unintentional

stop is a future HBM round trip and a future launch.

1.2.2 Layout and memory planning passes

Relax's graph passes include layout transformation and memory planning that run before TensorIR lowering, and their ordering determines whether a fused region materializes its intermediates (Lai *et al.*, 2023; Chen, 2020). A layout rewrite that runs after fusion can force the fused kernel to transpose through memory; memory planning that allocates one buffer per operator instead of reusing a single scratch region wastes HBM capacity without changing any arithmetic (Chen, 2026; Lai *et al.*, 2023). The destination-passing-style contract behind `call_tir` is what lets the planner see the whole fused region and allocate once (Lai *et al.*, 2023). Teams reviewing TVM pass changes should diff the generated TIR for buffer count and allocation size, not just for loop structure, because buffer regressions are the silent form of residency loss (Chen, 2026; Davies *et al.*, 2025).

1.2.3 Lowering checklist for one decode layer

Before accepting a TVM lowering for a decoder layer, check four artifacts: the fused operator count per layer in the Relax module, the generated TIR loop structure for any remaining per-operator buffers, the emitted kernel count on the target, and the captured or static launch plan that wraps those kernels (Chen *et al.*, 2018b; NVIDIA, 2024; AMD, 2024a). The same checklist applies on every target with target-specific wording: GPU uses kernels and command capture, CPU uses threads and cache tiles, NPU uses DMA edges and static tiles (AMD, 2024b; Goodfellow *et al.*, 2016; NVIDIA, 2022). If any of the four artifacts is missing, the lowering is incomplete regardless of how well the microbenchmark scores (Vellaisamy *et al.*, 2026; Chen, 2026). This checklist is short enough to be a CI gate and precise enough to catch the regressions that pass-level summaries hide (Chen, 2020; Davies *et al.*, 2025).

1.3 TVM codegen

Codegen is where the matrix meets Chapter 1's counters. TVM emits through several back-ends: LLVM for x86 and ARM CPUs, CUDA/NVRTC (or source handed to NVCC) for NVIDIA GPUs, ROCm for AMD, and Bring-Your-Own-Codegen (BYOC) to **emit** calls into vendor libraries or an NPU runtime for any **platform** TVM does not natively target (Chen *et al.*, 2018b; Chen, 2020). The

backend you pick decides the unit of work the host must launch.

The trap is to grade codegen by the quality of a single emitted matmul. At batch-1 decode the emitted MMA is rarely the limiter; the limiter is how many distinct kernels the host launches per token and how many HBM round trips survive lowering (Vellaisamy *et al.*, 2026; Chen, 2026). A backend that emits a beautiful per-operator kernel but leaves a decoder layer as a dozen separate launches loses to a coarser fused kernel that the runtime can capture once. On the GPU path this is why graph capture matters: replaying a captured launch sequence removes per-kernel host dispatch and recovered roughly 1.26× on a Qwen-2.5-7B batch-1 decode cell on H100 in the Memory-Floor study—without changing a single MMA tile (Chen, 2026; NVIDIA, 2024). On the CPU path the equivalent win is fusing through cache-resident tiles so the emitted code never spills the working set; on an NPU it is emitting a static DMA schedule rather than dynamic dispatch (AMD, 2024b,a).

The other codegen decision that decode telemetry punishes is the runtime that wraps the emitted kernels. TVM can package a model as a graph executor, a relax VM, or an ahead-of-time (AOT) module, and each carries a different per-token host cost (Chen *et al.*, 2018b; Lai *et al.*, 2023). A Python-driven graph executor adds interpreter and dispatch overhead on every operator—tolerable for batched prefill, fatal for batch-1 decode where that overhead is paid once per token with nothing to amortize it against (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023). The AOT path and a captured launch graph collapse that orchestration into a single replayable sequence, which is the codegen-level lever that actually moves ms/token (NVIDIA, 2024; Chen, 2026).

Multi-hardware delta. The CUDA backend can hide latency with asynchronous copy and double buffering, so its codegen can afford a dynamic schedule; the XDNA/AIE backend cannot, because its tiles and DMA descriptors are fixed before runtime (SemiAnalysis, 2024; AMD, 2024a). The same Relax module therefore emits genuinely different control flow per platform, and grading both against one microbenchmark hides the decode cost (Goodfellow *et al.*, 2016).

1.3.1 BYOC and accelerator codegen

Bring-Your-Own-Codegen is TVM's escape hatch for targets its native emitters do not cover, and it is the mechanism an XDNA/AIE or vendor-NPU port would use (Chen *et al.*, 2018b; AMD, 2024b). BYOC partitions the graph into regions a foreign compiler or runtime understands and emits calls into that runtime instead of

TVM kernels (Chen *et al.*, 2018b; Chen, 2020). For decode the partition boundary is the entire design problem: each boundary crossing is a host-call and a potential buffer copy, so the partitioner should prefer whole fused regions over per-operator handoffs (Davies *et al.*, 2025; Chen, 2026). A foreign runtime that only accepts prefill-shaped kernels will force the decode loop into a slow generic fallback unless the partitioner knows the decode shape (AMD, 2024b; Vellaisamy *et al.*, 2026). Teams should treat BYOC support as a contract with the same residency review as native codegen: which tensors stay inside the foreign region, which cross it, and what the crossing costs (Chen and YiRage Contributors, 2026; Chen, 2020).

1.3.2 Executor choice is a decode decision

The executor that drives emitted kernels is part of codegen for serving purposes. The classic graph executor walks a pre-serialized graph and dispatches operators through a runtime, which is flexible but pays dispatch cost per node; the Relax VM compiles the module to bytecode with a smaller runtime footprint; ahead-of-time compilation links the module into a standalone binary with the least per-call overhead (Chen *et al.*, 2018b; Lai *et al.*, 2023). For batch-1 decode, where one token step traverses the whole decoder graph, the per-node dispatch of the graph executor reappears as launch-like overhead on every token (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023). Measuring the executor cost requires comparing identical compiled kernels under each runtime at the decode shape; a difference that is invisible at batch 32 becomes the p99 floor at batch 1 (Davies *et al.*, 2025; Chen, 2026). Choose the executor with the serving loop, not with the training loop, in mind (Chen *et al.*, 2018b; Vellaisamy *et al.*, 2026).

1.3.3 Capture, driver, and artifact pinning

Whatever executor TVM uses, the GPU path should capture the compiled launch sequence so per-token host cost is paid once (NVIDIA, 2024; Chen, 2026). Capture is only stable when shapes, buffer addresses, and allocator behavior are stable, so the decode loop must be bucketed and the captured sequence regenerated at bucket boundaries (Kwon *et al.*, 2023; Davies *et al.*, 2025). The emitted kernels themselves depend on the TVM build, the CUDA driver, and the target description, so a serving artifact should pin all three and record them in the release manifest (Chen, 2020; Vellaisamy *et al.*, 2026). When the driver or TVM changes, the capture must be re-validated at every served context bucket before promotion (Chen, 2026; Chen and YiRage Contributors, 2026). This pinning is the TVM instance of the general compiler-deployment rule: the artifact that runs is the artifact that must

be reproduced (Davies *et al.*, 2025; Chen, 2026).

1.3.4 Host-device transfer boundaries

Decode graphs contain small tensors that some framework paths copy eagerly between host and device, and those copies are invisible in kernel-level benchmarks (Kwon *et al.*, 2023; Davies *et al.*, 2025). Sampling parameters, router outputs, and per-request metadata frequently cross the boundary once per token, adding latency that scales with round-trip count rather than tensor size (Vellaisamy *et al.*, 2026; Chen, 2026). TVM codegen should keep such values on-device wherever the control flow allows, and the serving integration should batch metadata updates outside the steady-state loop (Chen *et al.*, 2018b; Davies *et al.*, 2025). Stream ordering compounds the problem: if TVM emits kernels on a default stream while the runtime synchronizes per step, the synchronization defeats command-buffer capture (NVIDIA, 2024; Chen, 2026). The codegen review should therefore include the host-side driver code, not only the emitted device kernels, because the boundary is where decode latency hides after fusion is perfect (Chen, 2020; Davies *et al.*, 2025).

1.4 TVM tuning

TVM's auto-tuning lineage is the part most teams adopt and the part most likely to mislead. AutoTVM came first: a template-based tuner that learns a statistical cost model (gradient-boosted trees or TreeGRU) over hand-written schedule templates, uses transfer learning across workloads, and explores with simulated annealing—reporting 1.2–3.8× end-to-end gains over hand-tuned libraries on CPU, mobile GPU, and server GPU (Chen *et al.*, 2018a). Ansor removed the templates: it samples complete programs from a hierarchical search space, fine-tunes with evolutionary search and a learned cost model, and uses a task scheduler to spend **tun**ing budget where it moves end-to-end latency most (Zheng *et al.*, 2020). MetaSchedule later unified these search strategies over TensorIR.

The search budget is itself a hardware-bound decision. AutoTVM's transfer learning lets a cost model trained on one workload warm-start another, which matters because each candidate schedule must be compiled and measured on real silicon—thousands of trials per operator on a GPU, far more on a slow edge target (Chen *et al.*, 2018a). Ansor's task scheduler spends that budget where it changes end-to-end latency rather than spreading it uniformly, so a long-tail operator that dominates decode is not starved while a cheap one is over-tuned (Zheng *et al.*, 2020).

Table 1.1: Ansor template-free auto-scheduling: reported maximum end-to-end DNN speedups over prior search/library baselines, by hardware class (Zheng *et al.*, 2020).

Hardware class	Representative target	Max speedup
Server / desktop CPU	Intel x86 (AVX-512)	3.8×
Edge / mobile CPU	ARM (NEON)	2.6×
Server GPU	NVIDIA (CUDA)	1.7×

For a serving team the practical reading is that auto-tuning is not free wall-clock time; it is a measured search whose payoff depends on tuning the operators that actually appear on the batch-1 decode critical path.

The **pitfall** is the operating point. Auto-tuning maximizes whatever workload you **profile** against, and the default reflex is to tune on large, batched, prefill-shaped GEMMs because they are easy to benchmark and show big numbers (Kwon *et al.*, 2023). Schedules tuned at that shape over-tile for throughput and assume launch cost is amortized across many tokens—both assumptions break at batch-1 decode, where the GEMM is a thin matrix-vector product and the launch floor dominates (Vellaisamy *et al.*, 2026; Chen, 2026). The fix is mechanical: tune on the decode operating point (BS=1, realistic context), and rank candidates by measured ms/token after decomposing bytes/token and launches/token, not by the tuner's raw cost-model score on a synthetic shape (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). The same discipline differs by **hardware** class: GPU tuning chases coalescing and occupancy, CPU tuning chases cache locality and vector width, NPU "tuning" is mostly static tile assignment with little runtime freedom (AMD, 2024b; NVIDIA, 2022).

1.4.1 Search objective for decode

AutoTVM and Ansor measure candidate schedules on a reward that defaults to end-to-end latency for a given workload shape, and the entire search inherits whatever that shape implies (Chen *et al.*, 2018a; Zheng *et al.*, 2020). At the decode operating point, the reward should be composed from the same counters the rest of this book uses: bytes/token first, then launches or DMA edges per token, then measured ms/token at the served context bucket (Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). A schedule that minimizes a synthetic cost-model score on a batched shape will rank differently when the real objective is batch-1 memory traffic (Kwon *et al.*, 2023; Dao *et al.*, 2022). TVM lets users provide the measurement callback for search, so serving teams should wire the decode-cell

benchmark into the tuner rather than accepting the default benchmark builder (Chen *et al.*, 2018a; Zheng *et al.*, 2020). Search is only as honest as the workload it measures.

1.4.2 MetaSchedule and the modern stack

MetaSchedule unifies the earlier search strategies over TensorIR, providing program sampling, evolutionary search, and a learned cost model inside the current TVM workflow (Chen *et al.*, 2018b; Zheng *et al.*, 2020). For decode teams the relevant property is that MetaSchedule still searches within what the target description and pass pipeline allow: it can find a better tiling for an attention fusion, but it cannot fuse an operator that graph passes left outside the region (Chen *et al.*, 2018b; Davies *et al.*, 2025). The division of labor is therefore graph passes decide residency, MetaSchedule decides the best legal schedule for the residency that exists (Lai *et al.*, 2023; Chen, 2026). Upgrading TVM versions changes both layers, so a regression after an upgrade should be bisected between graph-pass output and schedule-search results before blaming either (Chen, 2020; Davies *et al.*, 2025).

1.4.3 Budget management and measurement

Autotuning budget is wall-clock time on real hardware, and the task scheduler exists precisely because operators are not equally important (Zheng *et al.*, 2020; Chen *et al.*, 2018a). The decode-critical operators are the ones on the per-token path: attention projections, KV-loop scores, softmax, output projection, and the MLP block (Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). Operators that run once per prefill or outside the steady-state loop deserve less budget (Davies *et al.*, 2025; Chen, 2026). The budget ledger should record trials per operator, the measured reward trend, and the final schedule fingerprint so a later search can resume rather than restart (Chen *et al.*, 2018a; Chen, 2020). Teams that treat tuning as an overnight job with no objective record cannot answer whether a later regression came from the search, the compiler, or the workload change (Vellaisamy *et al.*, 2026; Chen, 2026).

1.4.4 MoE-aware search constraints

Mixture-of-experts graphs distort tuning in two directions: the expert body multiplies the number of operators that could each be tuned, and data-dependent routing makes any single-shape measurement unrepresentative (Liu *et al.*, 2024; Kwon *et al.*, 2023). Search should constrain the expert region to a canonical schedule

tuned once and replicated, leaving budget for the router and the shared attention path (Davies *et al.*, 2025; Chen *et al.*, 2018b). Routing skew means the reward should be measured on request mixes, not on a single synthetic batch (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024). If the search cannot see routing distributions, its best schedule optimizes the average case the tuner invented rather than the fleet case (Chen, 2026; Davies *et al.*, 2025). The practical fix is to feed the tuner the serving trace and rank candidates by p99 over that trace, not by mean latency over a uniform synthetic set (Kwon *et al.*, 2023; Davies *et al.*, 2025).

1.4.5 Re-tuning cadence and upgrade triggers

Search results decay on the same triggers as every other compiler decision: TVM upgrades, target-description changes, driver updates, model architecture changes, and new context buckets all invalidate previous schedule fingerprints (Chen *et al.*, 2018b; Chen, 2026; Davies *et al.*, 2025). A lightweight nightly job should re-measure the pinned decode artifact and alert when bytes/token or launches/token drift beyond tolerance (Vellaisamy *et al.*, 2026; Chen, 2020). The alert triggers a bounded re-search over the affected operators with the previous schedule as a seed, not a full re-tune of the model (Zheng *et al.*, 2020; Chen *et al.*, 2018a). Full re-tunes belong on a release cadence with the artifact manifest updated atomically, so a bad search can be rolled back to the prior schedule in one step (Chen, 2026; Chen and YiRage Contributors, 2026). This cadence converts autotuning from an event into a maintained contract, which is the only model that survives framework churn and team turnover (Davies *et al.*, 2025; Chen, 2026).

1.5 TVM case study

Make the contradiction concrete with one decoder layer as the **case**. Tune it with Ansor on three targets and **benchmark** at the decode operating point, not the tuner's default (Zheng *et al.*, 2020; Vellaisamy *et al.*, 2026). The cross-hardware reading is consistent: per-operator schedules that win an isolated GEMM benchmark can still lose end-to-end because each operator is a separate launch and each unfused boundary is an HBM round trip (Chen, 2026; Dao *et al.*, 2022). The lever that moves p99 is fusing the layer so activations stay on-chip and capturing the launch sequence so the host stops paying per-kernel dispatch every token (NVIDIA, 2024; Vellaisamy *et al.*, 2026).

The arithmetic makes the lever concrete. Take a decoder layer at hidden size d=4096 in BF16, where each materialized activation vector costs $4096 \times 2 = 8\,\text{KB}$.

An unfused schedule that writes and re-reads roughly six such intermediates—layer-norm output, Q/K/V projections, attention output, MLP input—per token spends about 48 KB/layer/token of avoidable HBM traffic before the KV cache is even touched (Chen, 2026; Dao *et al.*, 2022). Fusing the layer so those vectors stay on-chip removes that traffic outright; at 32 layers it is on the order of 1.5 MB/token of HBM reads and writes that simply disappear, which is why bytes/token moves even when tensor-core utilization looks unchanged (Vellaisamy *et al.*, 2026). No tuner finds this if it is pointed at a single isolated GEMM.

This **cross-hardware** story is the book's recurring thesis: the optimization rank differs by silicon, so no single tuned artifact is portable in the way the build matrix suggests (Davies *et al.*, 2025). On the GPU the win is fusion plus graph capture; on the CPU it is cache-resident fusion and SIMD; on the XDNA NPU it is a static tile/DMA schedule with the fusion decided entirely at compile time (AMD, 2024b,a). **YiRage** operationalizes the case: it carries the Chapter 2 residency rules as checkable IR metadata, so a TVM-style schedule that would spill between graph and TensorIR levels is rejected at compile time across CUDA, CPU, and NPU back-ends rather than discovered in production telemetry (Chen and YiRage Contributors, 2026).

1.5.1 Layer schedule experiment

Run the layer experiment the way the earlier arithmetic suggests. Export one decoder layer, fuse it at the Relax level, schedule the fused region with MetaSchedule or Ansor at the batch-1 decode shape, and then compare three artifacts against an eager PyTorch baseline: the fused operator count, the emitted kernel or DMA-edge count, and bytes/token at the served context buckets (Chen *et al.*, 2018b; Zheng *et al.*, 2020; Davies *et al.*, 2025). The experiment should be repeated after changing the TVM version, the target string, or the driver, because each change can silently alter which fusions survive lowering (Chen, 2026, 2020). Record the winning artifact hash in the deployment manifest. The result of the experiment is not a speedup number but a reproducible claim that this layer, on this target, at this context, moves these counters (Vellaisamy *et al.*, 2026; Chen, 2026).

1.5.2 Numerical discipline inside search

Schedule search should not be allowed to change numerics silently. AutoTVM and Ansor explore loop orders, vector widths, and parallel strategies that are mathematically equivalent in the abstract but can differ in floating-point rounding, especially inside softmax, reductions, and mixed-precision graphs (Zheng *et al.*,

2020; Chen *et al.*, 2018a; Dao *et al.*, 2022). The reward function should include a bounded-error check against a reference implementation at the searched shapes, and candidates that exceed tolerance should be rejected regardless of speed (Dao *et al.*, 2022; Davies *et al.*, 2025). Mixed-precision search adds a second axis: allowing FP16 or BF16 accumulations enlarges the space and multiplies the numerical risk (Lai *et al.*, 2023; Chen, 2026). Compiler teams that skip the error term optimize schedules that may pass latency gates and fail quality gates in production (Chen, 2020; Vellaisamy *et al.*, 2026).

1.5.3 Operator ranking by counter impact

When the layer experiment shows a remaining gap, rank operators by which counter they bind. A layer-norm or residual chain that materializes each intermediate binds bytes/token and should be fused first; attention softmax state that crosses a reduction binds both bytes and launch count; the per-token GEMMs bind launch cost only when their host dispatch dominates (Dao *et al.*, 2022; Chen, 2026; Vellaisamy *et al.*, 2026). The ranking should come from the decode-cell profile, not from FLOPs, because FLOPs rank the GEMMs highest while the profile usually points at orchestration and memory seams (Davies *et al.*, 2025; Kwon *et al.*, 2023). After changing one operator's schedule, re-run the whole-layer benchmark rather than the operator microbenchmark, since the bottleneck may simply move to the next seam (Chen, 2026; Davies *et al.*, 2025). This ranking loop is the practical use of TVM's schedule power for decode: not tuning everything, but tuning the seams that move the counters (Chen *et al.*, 2018b; Chen and YiRage Contributors, 2026).

1.5.4 Practical decision rules

Four rules compress this chapter for a design review. First, compile the decoder layer as a named function, not the whole model, so fusion and capture apply to the unit the serving loop repeats (Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). Second, rank schedule investment by bytes/token seams before launches before FLOPs, and refuse cost-model scores that cannot be traced to measured counters (Chen, 2026; Davies *et al.*, 2025). Third, choose the executor and capture strategy by measuring their per-token host cost at batch-1 rather than by convenience of the training stack (Kwon *et al.*, 2023; NVIDIA, 2024). Fourth, pin the TVM version, target description, driver, and search artifacts together so any regression can be bisected to one change (Chen, 2020; Chen and YiRage Contributors, 2026). Teams that follow these rules get reproducible decode wins; teams that skip them get a compiler that was faster in the demo and slower in the fleet (Vellaisamy *et al.*,

2026; Chen, 2026).

1.5.5 Failure autopsies for TVM decode

TVM decode failures cluster into recognizable signatures. A schedule that wins isolated GEMM benchmarks yet loses whole-layer latency usually means fusion stopped at an import seam or a layout rewrite materialized an intermediate (Lai *et al.*, 2023; Chen, 2026). A win on one GPU that disappears on another usually traces to target strings and tile defaults that differ between SKUs, which artifact pinning exposes as a config diff (Chen *et al.*, 2018b; NVIDIA, 2022). A regression after a TVM upgrade should be bisected between graph passes and schedule search by dumping the module at both stages (Chen, 2020; Davies *et al.*, 2025). Each autopsy should start from the pinned artifact set and the emitted TIR diff, not from framework timers (Chen, 2026; Chen *et al.*, 2018b). The discipline turns every recurrence into a reproducible check instead of a folklore workaround (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

1.5.6 From one layer to the decoder loop

The layer experiment generalizes to the full decoder loop only when the loop structure survives compilation. A serving loop that calls the compiled layer function once per token should present a stable shape signature to TVM so each call reuses the same fused artifact and captured launch sequence (Dao *et al.*, 2022; Kwon *et al.*, 2023). If the loop instead grows the KV dimension dynamically per call, the runtime either falls back to a generic path or recompiles, and the per-token cost returns even though the layer artifact is perfect (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Bucket the KV length and compile one artifact per bucket, then measure the loop end-to-end including the host dispatch between calls (Chen, 2026; NVIDIA, 2024). The final claim for TVM in decode is therefore not a single-kernel speedup but an end-to-end artifact set: fused layer, captured or static launch plan, bucketed shapes, and a host loop that does not undo the compiler's work (Chen *et al.*, 2018b; Chen, 2026; Davies *et al.*, 2025).

1.5.7 Continuous batching and p99

Real decode serving rarely runs one request alone; continuous batching interleaves requests at different positions, which changes what the TVM artifact must optimize (Kwon *et al.*, 2023; Davies *et al.*, 2025). At batch one the compiler should minimize per-token latency and host dispatch; under continuous batching it must also

tolerate requests crossing context buckets and attention loops of different lengths inside one scheduler window (Vellaisamy *et al.*, 2026; Chen, 2026). The compiled artifacts remain per-bucket, so the runtime layer, not the compiler, owns the bucket handoff and the eager hull for dynamic requests (NVIDIA, 2024; Chen *et al.*, 2018b). Benchmarking should report p99 over the real request mix rather than mean latency on uniform synthetic data, because a few uncaptured fallback requests dominate tail latency exactly when they are invisible in the mean (Davies *et al.*, 2025; Kwon *et al.*, 2023). TVM tuning that stops at the kernel leaves this tail to the runtime; decode teams should tune the artifact and the scheduler together, and re-qualify the artifact set whenever the scheduler policy changes, because the compiler and the runtime jointly determine the p99 the product promises (Chen, 2026; Davies *et al.*, 2025). Document the request mix and the bucket-handoff policy beside the artifact manifest, so a tail regression is attributable to the scheduler, the compiler, or the workload rather than to a generic TVM slowdown (Kwon *et al.*, 2023; Davies *et al.*, 2025). Make the benchmark script part of the release so the tail number stays reproducible after every compiler and runtime change (Chen, 2026; Vellaisamy *et al.*, 2026).

1.6 Key Takeaways

- **The build matrix is not the decode matrix.** TVM lowers one tensor-expression definition to CPU, GPU, and accelerator back-ends, but a target that compiles is not automatically decode-legal (Chen *et al.*, 2018b). Validate each matrix entry against bytes/token and launches/token at batch-1, not against a per-operator microbenchmark (Vellaisamy *et al.*, 2026; Chen, 2026).

- **Co-design graph and TensorIR passes.** Relax graph fusion and TensorIR lowering live in one `IRModule` and must be optimized together; fusion that "succeeds" at the graph level still loses if lowering spills the fused producer to DRAM (Lai *et al.*, 2023; Chen and YiRage Contributors, 2026). Diff lowered IR for new allocations before merge (Chen *et al.*, 2018b).

- **Grade codegen by orchestration, not by one kernel.** At batch-1 the launch floor and surviving HBM round trips dominate, so a coarser fused-and-captured kernel beats a beautiful per-operator emit (Vellaisamy *et al.*, 2026; NVIDIA, 2024). Graph capture recovered roughly 1.26× on an H100 decode cell with no MMA change (Chen, 2026).

- **Tune at the decode operating point.** AutoTVM and Ansor maximize whatever shape you profile; tuning on prefill-shaped GEMMs over-tiles for

throughput and breaks at batch-1 (Chen *et al.*, 2018a; Zheng *et al.*, 2020). Profile BS=1 ms/token and decompose it before trusting a cost-model score (Davies *et al.*, 2025).

- **Optimization rank is hardware-specific.** GPU wants coalescing, fusion, and capture; CPU wants cache locality and SIMD; XDNA/AIE wants static tile and DMA assignment (AMD, 2024b,a). The same Ansor-tuned definition yields different legal schedules—and different speedups—per class (Zheng *et al.*, 2020).

1.7 Conclusion

TVM's compute/schedule split and its learning-based search remain the cleanest demonstration that performance portability is a compiler problem, not a library-porting problem (Chen *et al.*, 2018b; Zheng *et al.*, 2020). But the stack only pays off for interactive serving when its graph and TensorIR passes, its codegen back-end, and its auto-tuner are all pointed at the decode operating point and graded on bytes/token and launches/token (Vellaisamy *et al.*, 2026; Chen, 2026). That is the same residency contract YiRage enforces as IR metadata, and it is the bridge into the next compiler surface: Chapter **??** turns from TVM's whole-graph search to Triton's Pythonic, block-level GPU kernel compiler, where the same fusion-and-launch discipline reappears at the level of a single hand-written kernel (Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016).

Bibliography

AMD (2024a). Ai engine technology. https://www.AMD.com/en/products/adaptive-socs-and-fpgas/technologies/AI-engine.html. 2, 4, 5, 6, 7, 13, 17

AMD (2024b). AMD XDNA architecture. https://www.AMD.com/en/technologies/XDNA.html. 2, 3, 4, 5, 6, 7, 8, 10, 13, 17

Chen, J. (2026). Memory-bound but not bandwidth-limited: The physical AI inference gap in batch-1 LLM decode. *arXiv preprint arXiv:2605.30571*. 44-cell cross-GPU study; CUDA Graphs A/B on H100 and L4. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17

Chen, T., Zheng, L., Yan, E., Jiang, Z., Moreau, T., Ceze, L., Guestrin, C., and Krishnamurthy, A. (2018a). Learning to optimize tensor programs. In *Advances in Neural Information Processing Systems (NeurIPS)*. AutoTVM; template-based statistical cost model (GBT/TreeGRU), transfer learning; 1.2–3.8x end-to-end. 9, 10, 11, 12, 14, 17

Chen, T., Moreau, T., Jiang, Z., Zheng, L., Yan, E., Cowan, M., Shen, H., Wang, L., Hu, Y., Ceze, L., Guestrin, C., and Krishnamurthy, A. (2018b). TVM: An automated end-to-end optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 578–594. Graph- and operator-level optimization, operator fusion, memory latency hiding, ML-based cost model; CPU/GPU/FPGA. 1, 2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 13, 14, 15, 16, 17

Chen, X. (2020). Compiler-based hardware profiling for software-defined architecture. 1, 2, 3, 4, 5, 6, 8, 9, 11, 12, 13, 14, 15

Chen, X. and YiRage Contributors (2026). Yirage: Multi-backend LLM inference optimization. https://github.com/chenxingqiang/YiRage. Yield Revolutionary AGile Engine; extends Mirage with CUDA, CPU, MPS, Triton, NKI, and MLIR backends. 2, 3, 4, 5, 8, 12, 13, 14, 15, 16, 17

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*. 3, 5, 10, 11, 12, 13, 14, 15

Davies, M. *et al.* (2025). Liminal: Exploring the frontiers of LLM decode performance. *arXiv preprint arXiv:2507.14397*. Limit study; DeepSeek-V3 provisions 10× more decode than prefill nodes. 1, 3, 4, 5, 6, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. 2, 6, 7, 10, 17

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*. 1, 3, 7, 8, 9, 10, 11, 12, 14, 15, 16

Lai, R., Shao, J., Feng, S., Jin, Y., *et al.* (2023). Relax: Composable abstractions for end-to-end dynamic machine learning. *arXiv preprint arXiv:2311.02103*. TVM Unity; cross-level IRModule (Relax graph + TensorIR), first-class symbolic shapes, call_tir DPS; replaces Relay. 2, 3, 4, 5, 6, 7, 8, 11, 14, 15, 16

Liu, A. *et al.* (2024). Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*. 11, 12

NVIDIA (2022). Nvidia hopper architecture in-depth. NVIDIA Technical Blog. 2, 6, 10, 15

NVIDIA (2024). Cuda graphs. CUDA Programming Guide. 1, 5, 6, 7, 8, 9, 12, 14, 15, 16

SemiAnalysis (2024). Nvidia tensor core evolution: From volta to blackwell. Newsletter. 2, 7

Vellaisamy, P., Tripathi, S., Natarajan, V., Thenarasu, S. S., Blanton, S., and Shen, J. P. (2026). Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition. *arXiv preprint arXiv:2603.12465*. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17

Zheng, L., Jia, C., Sun, M., Wu, Z., Yu, C. H., Haj-Ali, A., Wang, Y., Yang, J., Zhuo, D., Sen, K., Gonzalez, J. E., and Stoica, I. (2020). Ansor: Generating high-performance tensor programs for deep learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 863–879. Template-free auto-scheduler; hierarchical sampling, evolutionary search, task scheduler; up to 3.8x/2.6x/1.7x on Intel CPU/ARM CPU/NVIDIA GPU. 9, 10, 11, 12, 13, 17

Index

Apache TVM, 1

TensorIR, 1
TVM, 1